

CS112: Friday 4 September, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

Wednesday ended on `&x`, a number printed in hexadecimal that nobody had explained yet, after types, sizes, uninitialized memory, and the three bases.

Today: memory is numbered boxes, and a pointer holds one of those numbers. Once that is on the board, arrays stop being a separate feature of the language. `a[i]` is not a lookup, it is arithmetic, and the square brackets are shorthand for a calculation you could write yourself.

PART

1. Address-of and dereference
2. An array is contiguous
3. Pointer arithmetic

Part 1. Address-of and dereference

```
int y = 7;
int *p = &y;           // p holds the ADDRESS of y

cout << p << endl;     // 0x7ffd... where y lives
cout << *p << endl;    // 7         what is there
*p = 99;               // change y without naming it
cout << y << endl;     // 99
```

Two symbols, and they are opposites. `&` asks where, `*` asks what is there.

Talk to your neighbor. What does `*y` mean, when `y` is an `int`?

Nothing, and it is a compile error. Dereferencing only makes sense on something that holds an address.

Part 2. An array is contiguous

```
int arr[5] = {10, 20, 30, 40, 50};

cout << arr      << endl;      // an address
cout << &arr[0] << endl;      // the same address
cout << &arr[1] << endl;      // four bytes further on
```

The array's name is the address of its first element. The elements sit in a row with nothing between them, each exactly `sizeof(int)` wide.

Three facts that follow, and all three bite later: the size is fixed, nothing checks your index, and the contents start as garbage.

Part 3. Pointer arithmetic

```
int *ptr = arr;
cout << *(ptr + 1) << endl;    // 20
cout << arr[1]      << endl;    // 20
```

Why does `+ 1` move four bytes and not one?

Because the compiler knows the type, so it counts in elements, not bytes. The identity goes on the board:

$$a[i] \equiv *(a + i)$$

The brackets are notation. The machine does the addition either way.